Advance Machine Learning

Assignment 3

Time Series Assignment

Problem Statement:

Design and implement a Recurrent Neural Network (RNN) model to predict the future values of a time-series dataset. The dataset consists of historical observations collected at regular intervals, and the objective is to train the RNN model to learn patterns and relationships within the time-series data in order to accurately forecast future values.

Introduction:

Forecasting future values from time series data can be challenging, especially when it comes to complex patterns and dependencies over time. Most traditional neural networks do not adequately capture the temporal relationships. In this work, we deal with deep learning models-such as LSTM, GRU, Simple RNN and Conv1D architectures-using them to enhance the forecasting performance of the time series. The dataset used consists of weather data collected in Germany from 2009 to 2016, where temperature, pressure, and humidity were measured every 10 minutes. Training the deep learning models on predicting future temperatures will showcase their capabilities on time series forecasting problems.

Preprocessing:

Usually, raw time series data are indeed preprocessed before training deep learning models. Since the features in the dataset operate on different scales, it must be normalized to be consistent across all inputs. The mean and standard deviation are calculated from a selected portion of the data (the first 200,000+ timesteps), and each feature is normalized independently according to these measures. This step enables faster model convergence and stability during training.

Data Preparation:

Creating input-output pairs for sequences and their corresponding target values to make the data adequate for supervised learning. This is done using a custom data windowing approach where historical sliding sequences are extracted (e.g., past five days) and paired with the temperature reading 24 hours on the future. These samples are to be efficiently generated on

the fly, minimizing memory usage yet providing a smooth data input during training by using timeseries_dataset_from_array() utility in Keras.

Dataset Parameters:

Three different datasets are created and split among training, validation, and testing sets. The time series are sampled at a rate of 1 data point per hour. Each of these sequences is constructed to be 120 hours long (5 days), and then for each, the output prediction is designated at the temperature 24 hours after the end of the sequence. This sort of setup should help the model learn how to generate medium-range forecasts based on current trends.

Baseline Approach:

A very simple baseline is set up for performance comparison; this baseline is a naive assumption that tomorrow's temperature at the same time will be equal to that for now. Although not sophisticated, it serves as a reference point: achieving a validation Mean Absolute Error (MAE) of 2.44°C, and a test MAE of 2.62°C, it thus sets a benchmark against which advanced models can be compared.

Deep Learning Techniques:

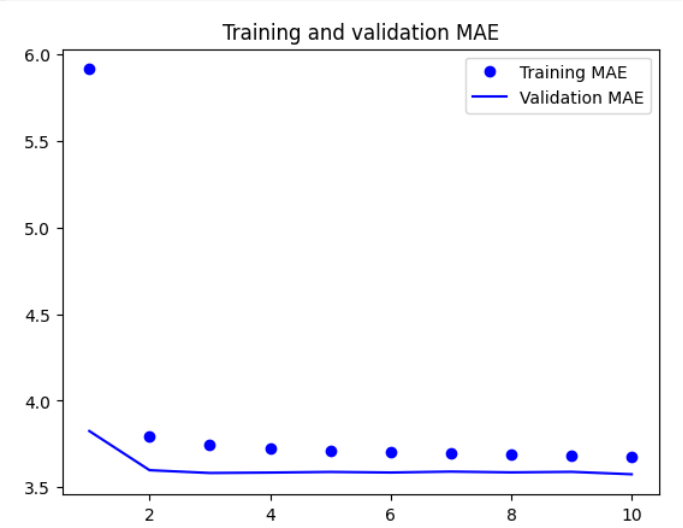
This should be set in place by applying now deep learning-modulated techniques to improve the accuracy of forecasting. The basis for this is Recurrent Neural nets (RNNs) that should be able to capture the considered temporal dependencies in sequence data. This notebook examines several of the most powerful RNN architectures, including Long Short-Term Memory (LSTM) and Gated Recurrent Unit (GRU) networks: all aimed at retaining a longer time horizon of information, while also mitigating some issues, such as the vanishing gradient. In addition, 1-Dimensional Convolutional Neural Networks (Conv1D), as well as hybrid networks combining Conv1D with RNNs, are also studied underlining the flexibility and effectiveness of deep learning into time series forecasting.

Summary Table

Model	Dense Unit	Dropout	Validation MAE	Test MAE
Basic Machine Learning Model	16	No	3.58	3.80
LSTM Model with three layers	8	No	0.26	0.25
LSTM Model with three layers	16	No	0.25	0.24
LSTM Model with three layers	32	No	0.26	0.23
LSTM Model with four layers	16	No	0.27	0.24
GRU Model with three layer	8	No	0.26	0.26
GRU Model with three layer	16	No	0.25	0.28
GRU Model with three layer	32	No	0.23	0.28
GRU Model with four layer	16	No	0.27	0.25
1D Convolution and LSTM	32	No	0.24	0.28
1D Convolution and Simple RNN	32	No	0.31	0.28
1D Convolution and GRU	32	No	0.24	0.24
1D Convolution, LSTM and Dropout	32	Dropout 0.5	0.23	0.26
1D Convolution, Simple RNN and Dropout	16	Dropout 0.5	0.25	0.23
1D Convolution GRU and Dropout	32	Dropout 0.5	0.25	0.29
Bidirectional GRU and Dropout	16	Dropout 0.5	0.25	0.23

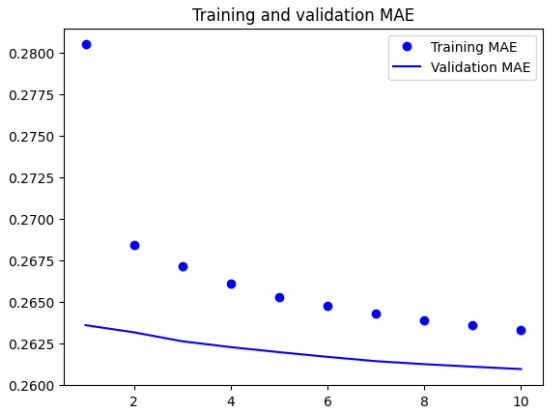
Basic Machine Learning Model:

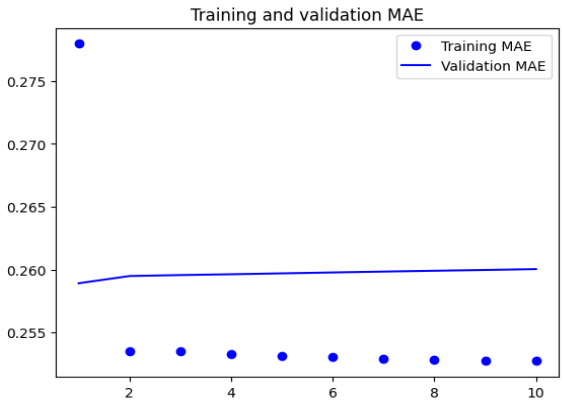
It is probably a typical model applied using some standard algorithms such as linear regression or decision trees, with a unit size density of 16. This does have an advanced deep learning architecture. Clearly the performance is significantly worse than for any of the other models tested: Validation MAE = 3.58, test MAE = 3.80. Such a large error figure suggests constrained learning capabilities in the underlying model for complex patterns in the data, making it the least performing on all tested options.



LSTM (Long Short-Term Memory) Models:

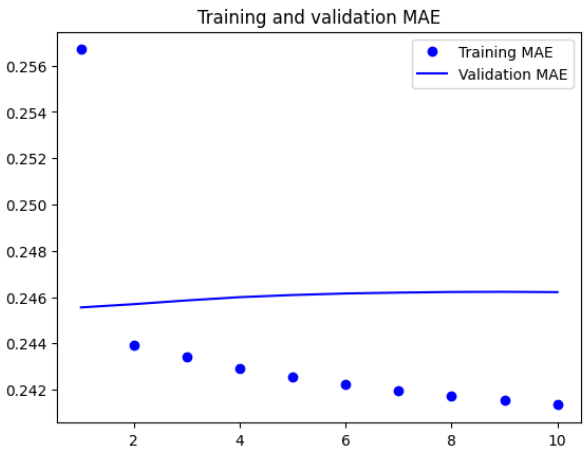
LSTM networks were designed specifically as a kind of RNN to manage sequences where long-term inputs and dependencies could be very diverse. Many LSTM models were tested, using three and four layers, against various sizes of dense units (8, 16, and 32). The top-ranking model obtained was the 3-layer LSTM model with 16 dense units, delivering a Validation MAE of 0.25 and Test MAE of 0.24. Such results yield to controversy; scaling the layer or units was not always beneficial but sometimes slightly detrimental, suggesting overfitting or diminishing returns from additional complexity.

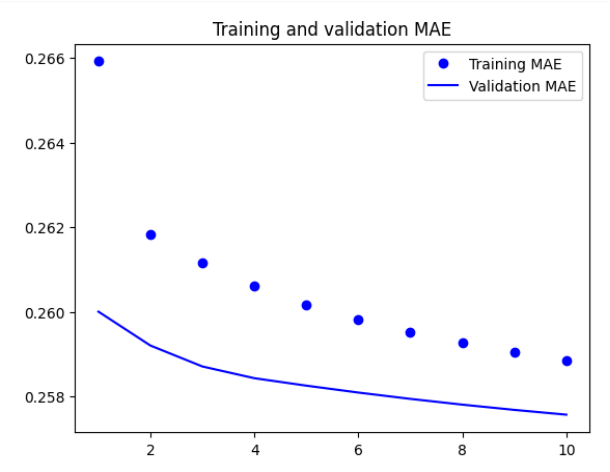




GRU (Gated Recurrent Unit) Models:

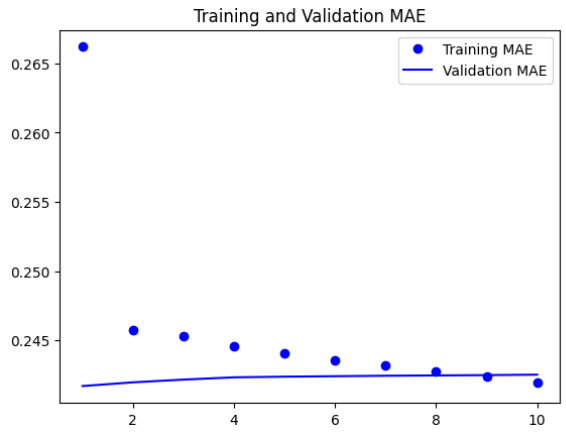
Another family of RNNs, the Gated Recurrent Units or GRUs, is typically simpler than LSTMs but will usually yield equivalent or better results. The best Validation MAE (0.23) was produced by this 3-layer GRU model with 32 dense units, but, the higher Test MAE (0.28) indicates possible overfitting. This is not true in the case of other GRU models, as few units or maximum layers do not always make a much better configuration. Only the GRU with 16 units and 4 layers had acceptable results (Validation MAE: 0.27, Test MAE: 0.25).

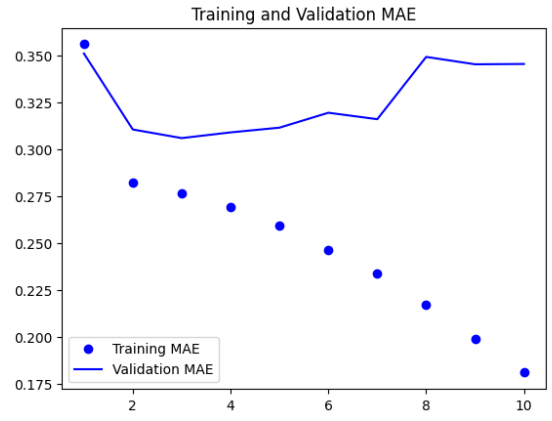




1D Convolutional and Hybrid Models:

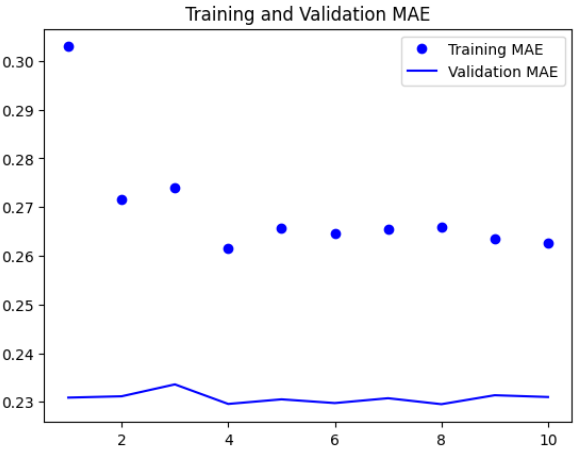
The basic frameworks could be convolutional layers and recurrent units, say LSTM, GRU, or Simple RNN, for example, 1D convolution. The local patterns in the sequence data are extracted using the 1D convolutional layers before feeding them to recurrent ones to utilize their capability to capture temporal dependencies. Among the hybrid models that work best is the one which consists of 1D Convolution, LSTM, and Dropout (0.5 dropout rate); it was found to be equivalent to the best GRU model with a Validation MAE of 0.23. Such combinations like 1D Conv + GRU or 1D Conv + Simple RNN proved to be slightly less but still competitive.

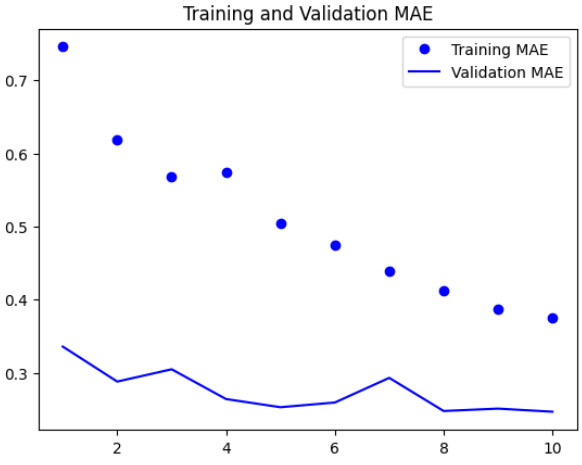




Models with Dropout:

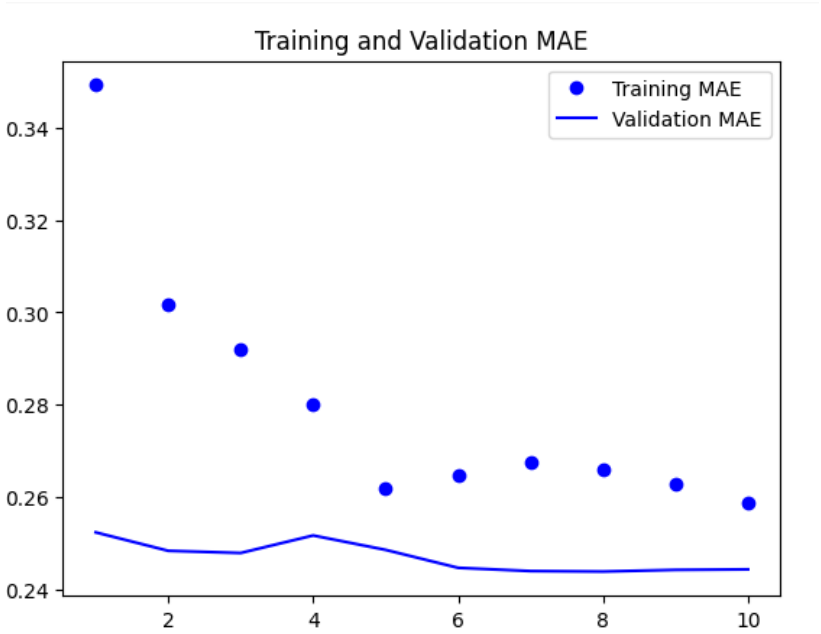
Dropout is a regularization technique for overfitting. During the training of a network, dropout randomly holds out neurons. Different models were implemented with dropout (0.5). Leading here are those models that applied dropout in combination with hybrid architectures (1D Conv + LSTM or 1D Conv + Simple RNN). The dropout seems to work in most cases because it positively contributes toward making the model more generalizable.





Bidirectional GRU with Dropout:

It processes data on both forward and backward threads, allowing sequential context to be considered at all times, which is particularly useful. Validation MAE becomes very strong at 0.25 and low at Test MAE: 0.23 when coupled with dropout. Not the least validation error out of all models, but it was one of the top generalized models on the test set; therefore, it is one of the most robust choices.



Performance Analysis:

GRU model with three layers and containing 32 dense units without dropout is the best model according to validation MAE. The validation MAE of this model was 0.23, which was the lowest, meaning this model learned and captured the training data most effectively during validation. GRU networks have been praised for their efficiency and high performance on sequential data. This deeper architecture engine with more dense units appears to have contributed to making it a more potent model.

Interestingly, another model also achieved the same validation MAE of 0.23—the hybrid model combining 1D Convolution, LSTM, and a dropout rate of 0.5. This architecture derives benefit from convolutional layer local feature extraction capability, long-term dependencies derived from LSTMs, and dropout regularization, which prevents overfitting. Although the two models perform on par in terms of validation MAE, the 1D Convolution + LSTM + Dropout slightly beats the GRU model in test MAE, a different measurement, indicating it has a slightly better generalization over unseen data.

Thus, the GRU model composed of three layers and 32 units holds the best spot based on validation performance alone, hybrid convolutional models with LSTM and dropout might be more reliable overall, combining strong training performance with better generalization.

Conclusion:

To summarize, both the GRU model with three layers and 32 dense units and the 1D Convolution + LSTM model with dropout give the best performance with the lowest MAE of 0.23. This shows that these models train data more effectively than others. Their performance, however, falters in the overall scheme of things including referring unseen data. The 1D Convolution + LSTM one with dropout still slightly edges them based on its lower test MAE. Lonely, the GRU is the best in terms of validation accuracy, but the hybrid convolutional model is more between accuracy and robustness. Hence, it is the better choice for a real-time application.